SEMANA 16 — INTELIGENCIA ARTIFICIAL EN LA NUBE III

IA en la Nube III: Capacidades y Repaso Unidad 3

Parte 1 de 4 — Modelos de Razonamiento Profundo e Inferencia

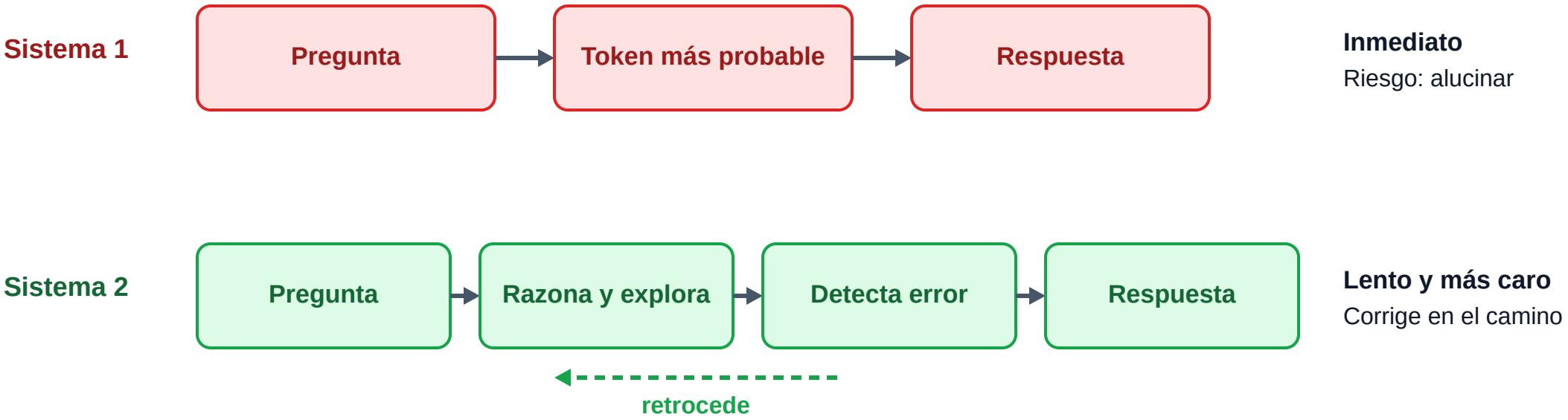
Ing. Rodolfo Cañas Cervantes
Universidad de la Costa (CUC) — Ingeniería de Sistemas
Semana 16 — Periodo 2026-2 — Barranquilla, Colombia

Parte 1 de 4 — Modelos de Razonamiento Profundo e Inferencia

Del Reconocimiento de Patrones al Razonamiento

De respuestas intuitivas inmediatas a procesos deliberativos de deducción lógica paso a paso

Dos formas de llegar a una respuesta



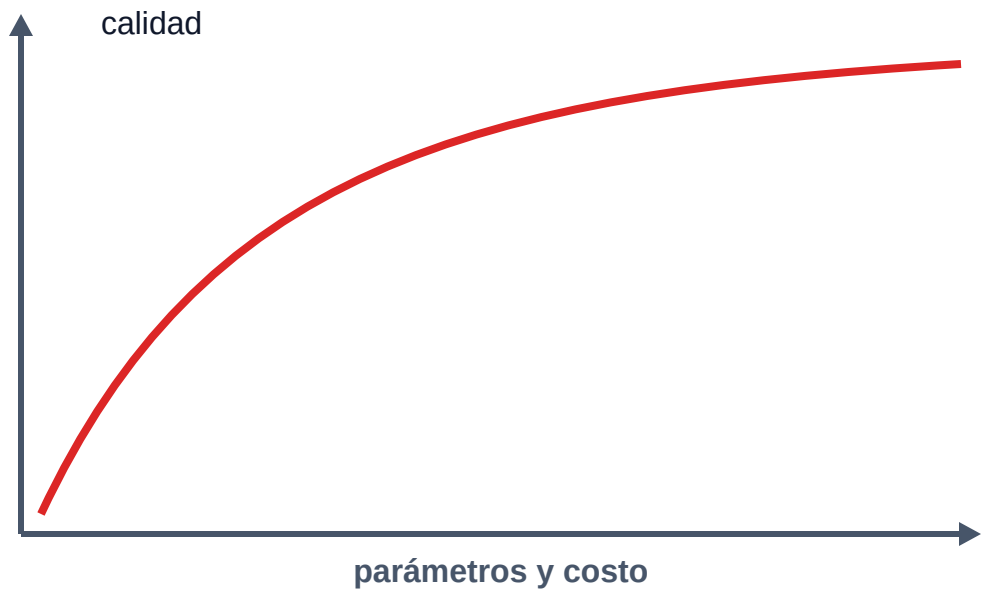
Ejemplos: LLM autorregresivo clásico frente a modelos con razonamiento como DeepSeek-R1

Fuente: OpenAI — Learning to Reason with LLMs (Research Report 2024)

Escalado del Cómputo en Tiempo de Prueba

Invertir más tokens durante la inferencia incrementa la precisión sin agrandar el modelo

Más parámetros: retorno decreciente



Más tiempo de razonamiento

Modelo 405B — **sin razonar**

Modelo 32B — **con razonamiento lo supera**

El costo pasa del entrenamiento
al servicio de la API

La inteligencia también se compra por consulta, no solo al entrenar

Fuente: Snell et al. — Scaling LLM Test-Time Compute (UC Berkeley & Google DeepMind 2024)

MECANISMO INTERNO

Cadena de Pensamiento Oculta (Hidden CoT)

Generación de trazas de razonamiento no visibles para el usuario final mediante etiquetas de control

1. Prompt de Entrada

Problema Complejo
Requisitos de arquitectura y red

Sin Instrucción CoT
No requiere 'piensa paso a paso'

Comportamiento Nativo
El modelo decide cuándo reflexionar

2. Bloque <think>

Tokens de Pensamiento
Genera cientos a miles de tokens

Exploración y Descarte
Prueba casos límite y errores

Etiqueta de Cierre
Termina con </think>

3. Respuesta Visible

Síntesis Ejecutiva
Respuesta pulida y estructurada

Alta Consistencia
Sin divagaciones ni errores

Facturación en Nube
Los tokens <think> se cobran

Fuente: DeepSeek-R1 Technical Report & OpenAI Reasoning Architecture (2025)

Árboles de Pensamiento y Backtracking

Exploración sistemática de caminos alternativos y retroceso ante callejones sin salida lógicos

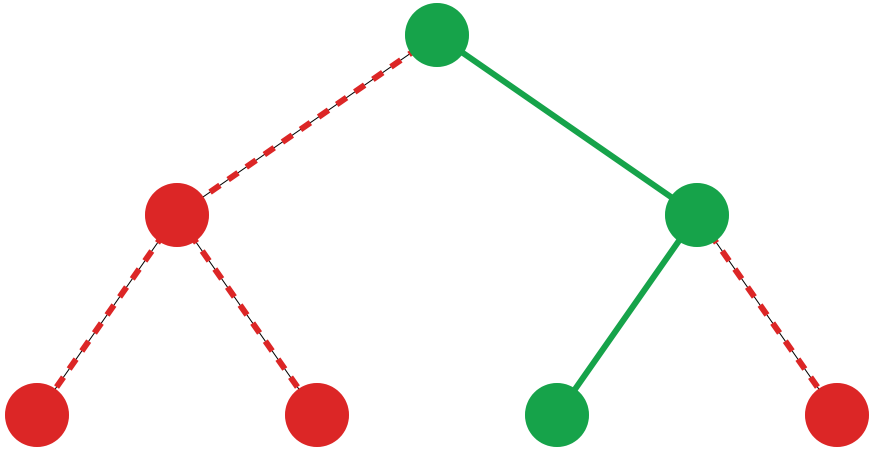
Greedy: una sola ruta



error inicial

y persiste en él

Árbol: explora y poda



Rojo: rama podada; retoma el camino válido

Volver atrás es lo que evita el efecto bola de nieve

Fuente: Yao et al. — Tree of Thoughts: Deliberate Problem Solving with Large Language Models (2024)

Entrenamiento sin Supervisión Humana: GRPO

Optimización por políticas de grupo con recompensas basadas en reglas verificables

1. Muestreo de Grupo

Generación Múltiple
Genera N respuestas (ej. N=8)

Diversidad de Trazas
Diferentes caminos deductivos

Cero Humanos en Bucle
No requiere anotadores humanos

2. Función de Recompensa

Verificación Binaria
Compilación y cálculo exacto

Recompensa de Formato
Premia el uso estricto de <think>

Comparativa Relativa
Normaliza ventajas dentro del grupo

3. Actualización de Pesos

Refuerzo de Trazas Válidas
Refuerza caminos óptimos

Sin Modelo Crítico
Ahorra 50% de VRAM en training

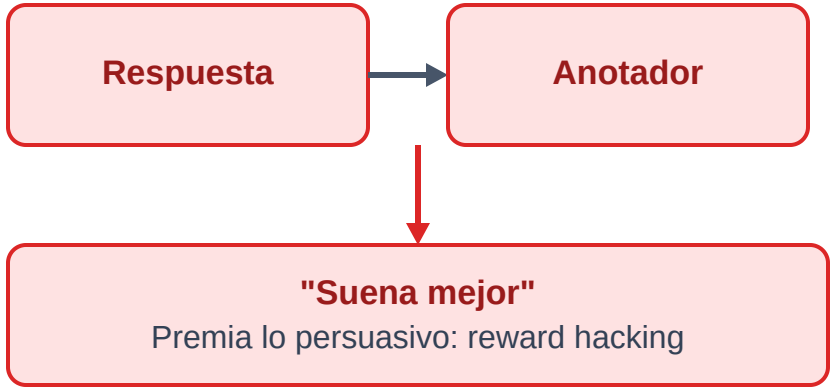
Aparición de Auto-Reflexión
El modelo aprende solo a verificar

Fuente: Shao et al. — DeepSeekMath & DeepSeek-R1: Group Relative Policy Optimization (2025)

Verificación Determinista y Recompensas

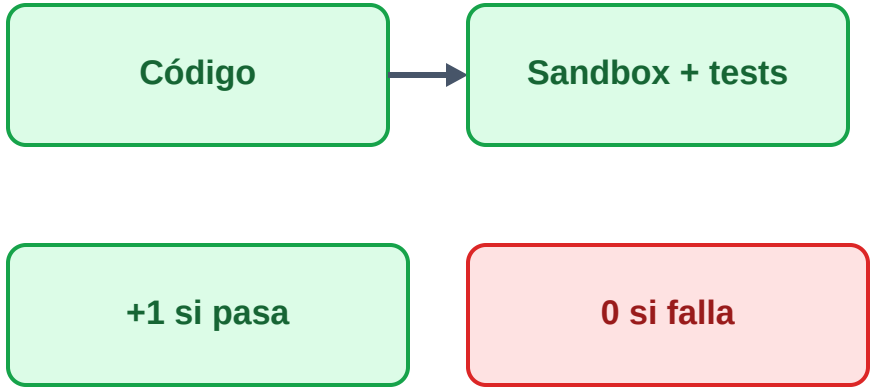
Sustitución de juicios humanos subjetivos por compiladores y evaluadores formales de software

Recompensa por preferencia humana



Costoso, lento y no escala

Verificador determinista



Premia la corrección, no la retórica

Cuando la respuesta se puede comprobar, la máquina es mejor juez

Fuente: Lambert et al. — Reward Modeling and Rule-Based Verification in LLMs (HuggingFace 2024)

Anatomía de los Tokens de Razonamiento

Diferenciación estricta entre tokens de pensamiento (ocultos) y tokens de respuesta (visibles)

Lo que el usuario ve no es lo que se factura



Total = entrada + thinking + visibles

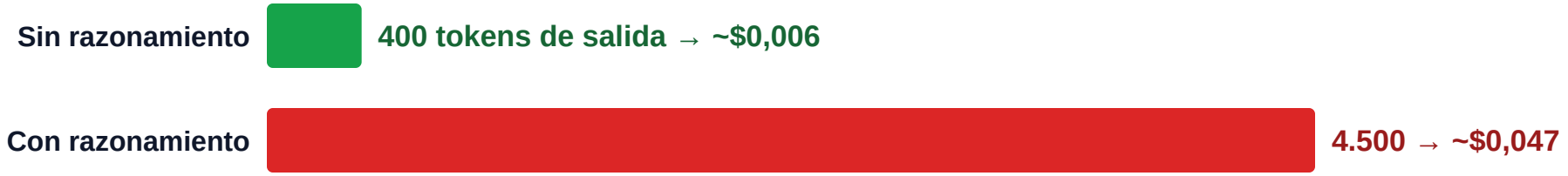
Controlar el razonamiento es controlar la factura

Fuente: Amazon Bedrock API Documentation — Reasoning Tokens Breakdown (2025)

Costos y Retardos en Entornos de Producción

Trade-off directo: mayor calidad cognitiva a cambio de mayor latencia TTFT y costo por llamada

Ejemplo de cálculo: Sonnet 5.5 ($2 entrada / $10 salida) y 1.000 tokens de entrada



Tiempo hasta el primer token visible



Unas 8 veces más costo y mucha más espera: reservarlo para lo que lo merece

Fuente: AWS Bedrock Performance Benchmarks & Real-World Pricing Analysis (2025)

Control de Presupuesto de Razonamiento

Parámetros de API para regular la profundidad del pensamiento según el SLA y el costo

1. Presupuesto Bajo (Low)

Límite de Tokens
budget_tokens: 1.024 tokens

Perfil de Latencia
Latencia reducida (< 5 seg)

Uso Recomendado
Consultas interactivas moderadas

2. Presupuesto Medio (Medium)

Límite de Tokens
budget_tokens: 4.096 tokens

Perfil de Latencia
Latencia moderada (10 a 20 seg)

Uso Recomendado
Refactorización y revisión de PRs

3. Presupuesto Alto (High)

Límite de Tokens
budget_tokens: 16.384+ tokens

Perfil de Latencia
Ejecución asíncrona (> 45 seg)

Uso Recomendado
Diagnóstico crítico y demostraciones

Fuente: Anthropic API — Extended Thinking & Reasoning Budget Configuration (2025)

Auto-Corrección y Debate Multi-Agente

Enfrentamiento dialéctico entre agentes especializados para erradicar sesgos y errores lógicos

1. Agente Proponente

Generación Inicial
Propone diseño o solución de código

Razonamiento Base
Justifica las decisiones adoptadas

Sesgo Típico
Puntos ciegos no anticipados

2. Agente Crítico (Red Team)

Búsqueda de Fallos
Revisa fallos y condiciones carrera

Desafío Dialéctico
Exige pruebas y contraejemplos

Rol Antagónico
Incentivado a encontrar errores

3. Agente Juez (Consenso)

Evaluación Imparcial
Evalúa argumentos del debate

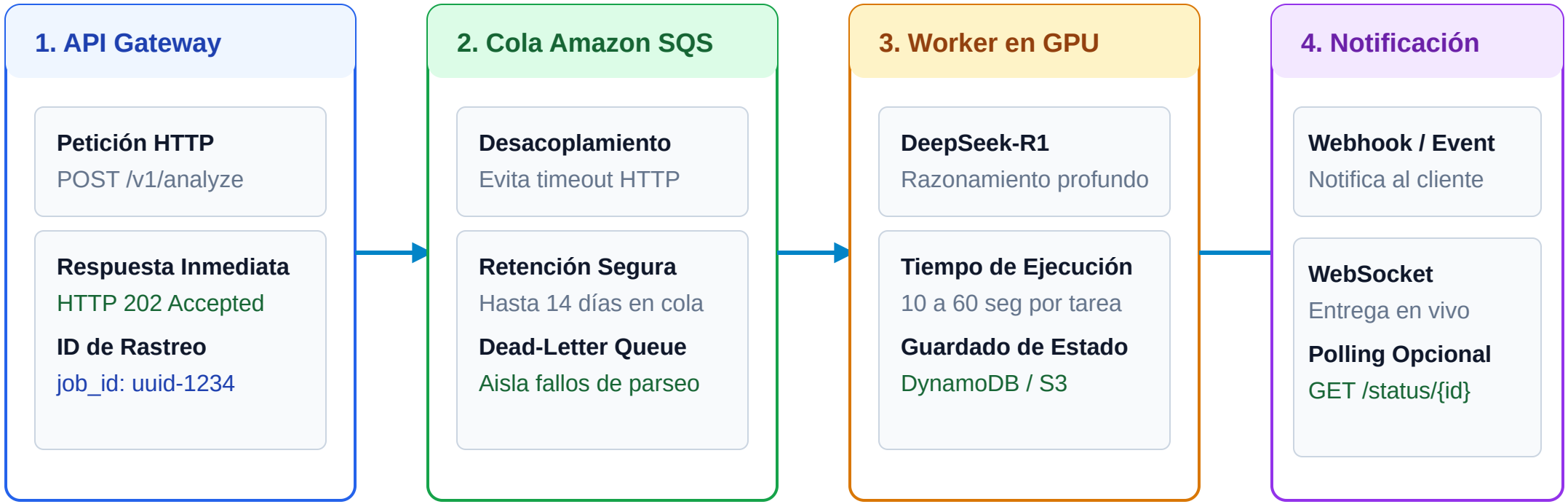
Refinamiento Final
Sintetiza la solución blindada

Resultado Robusto
Tasa de alucinación cercana a cero

Fuente: Du et al. — Improving Factuality and Reasoning in Language Models through Multiagent Debate (2024)

Arquitectura de Servicio para Modelos de Razonamiento

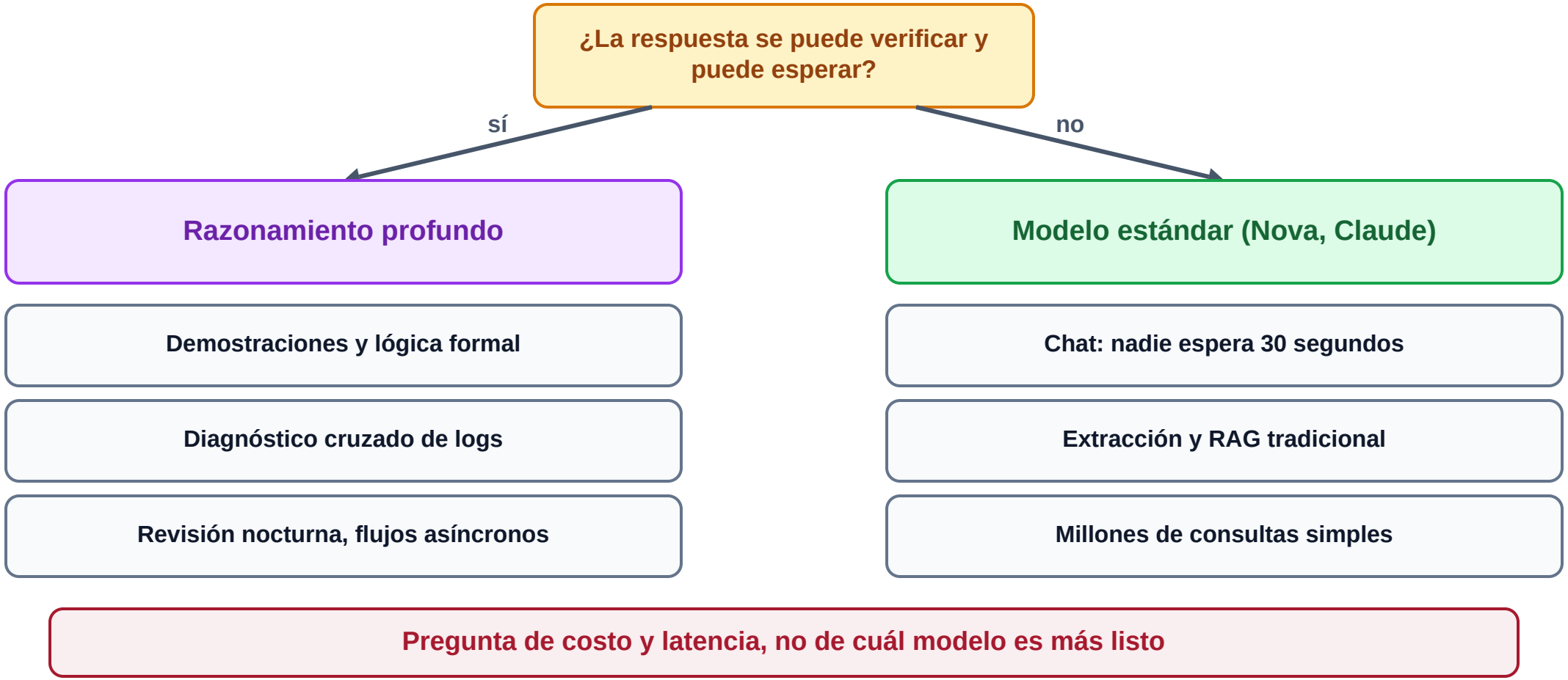
Patrones asíncronos y desacoplados con colas SQS y Webhooks para absorber largas esperas



Fuente: AWS Architecture Center — Asynchronous Generative AI Pattern (2025)

¿Cuándo Usar Razonamiento Profundo?

Matriz de selección: alta complejidad lógica justifica la latencia; tareas rutinarias exigen inmediatez

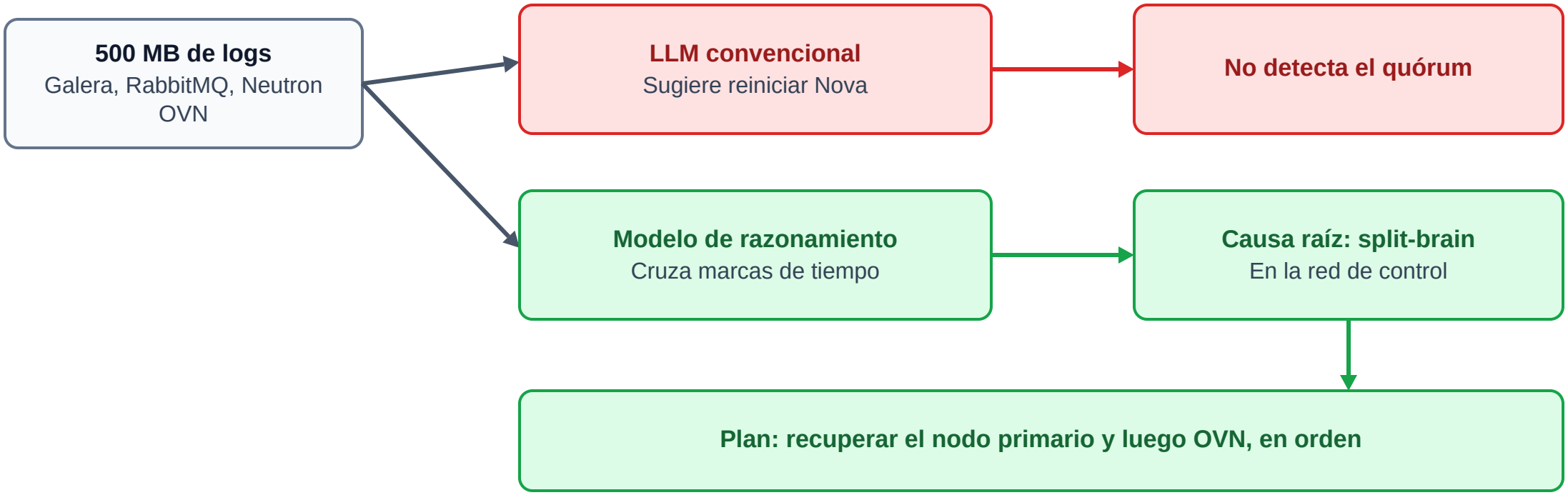


Pregunta de costo y latencia, no de cuál modelo es más listo

Fuente: Guía de Selección de Modelos Cognitivos — Universidad de la Costa (2026)

Caso Ilustrativo: Diagnóstico en Nube Privada

Caso ilustrativo de arquitectura, no es una empresa real — Resolución de fallos en OpenStack



Caso ilustrativo: el valor está en correlacionar, no en responder rápido

Fuente: Metodología de Diagnóstico de Infraestructura — Laboratorio CUC (2026)

CHECKLIST DE ARQUITECTURA

Principios Clave de Razonamiento en la Nube

Cuatro directrices fundamentales para integrar modelos cognitivos en producción

1. Flujo Asíncrono

Desacoplamiento

Diseñar con colas SQS y WebSockets para absorber tiempos de espera de reflexión

Evitar timeouts

2. Control FinOps

Presupuesto Tokens

Configurar topes de budget_tokens para contener costos de los tokens de think

Límites estrictos

3. Verificación

Pruebas Reales

Combinar deducción con ejecutores de test y compiladores en sandboxes aislados

Cero alucinación

4. Ruteo Híbrido

Gateway de Decisión

Despachar solo el 20 % de tráfico complejo a modelos de razonamiento y 80 % a MaaS ligero

Máximo ROI

SEMANA 16 — INTELIGENCIA ARTIFICIAL EN LA NUBE III

Programación Asistida y Micro-Sandboxing

Parte 2 de 4 — Generación de Código, Evaluación en SWE-bench y Aislamiento en Nube

Ing. Rodolfo Cañas Cervantes

Universidad de la Costa (CUC) — Ingeniería de Sistemas

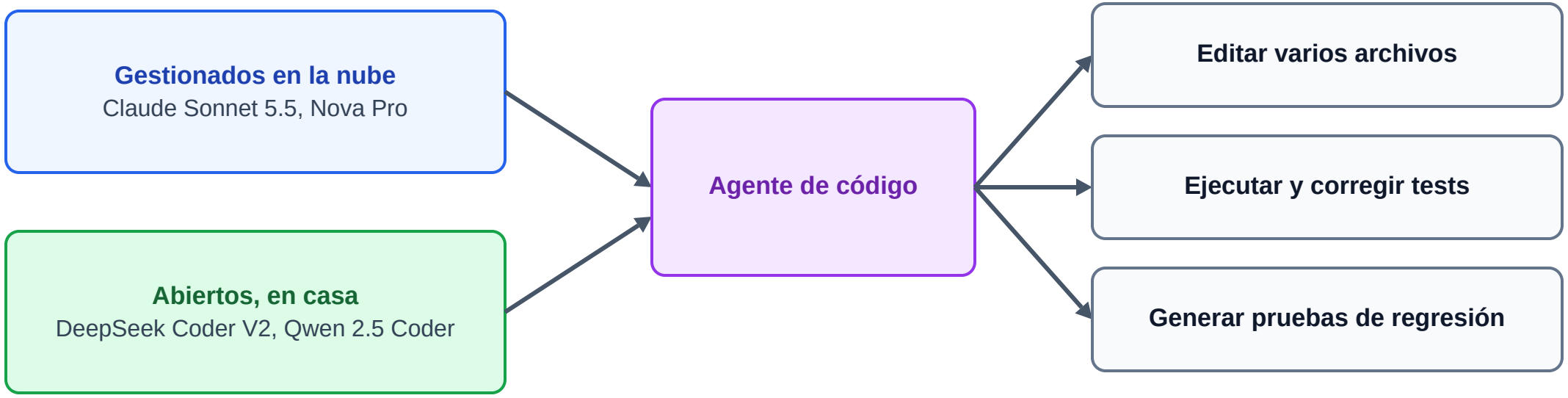
Semana 16 — Periodo 2026-2 — Barranquilla, Colombia

Parte 2 de 4 — Generación de Código, Evaluación en SWE-bench y Aislamiento en Nube

Modelos Especializados en Código

Entrenamiento con sintaxis formal, pruebas unitarias y repositorios versionados

Del modelo a la tarea: qué se elige



Importa menos el nombre del modelo que lo que logra en un repositorio real

Fuente: Anthropic / Meta AI — Code Generation Technical Reports (2024)

Evolución de Benchmarks: De Funciones a Repositorios

De acertijos sintéticos aislados a resolución de incidencias en proyectos reales de GitHub

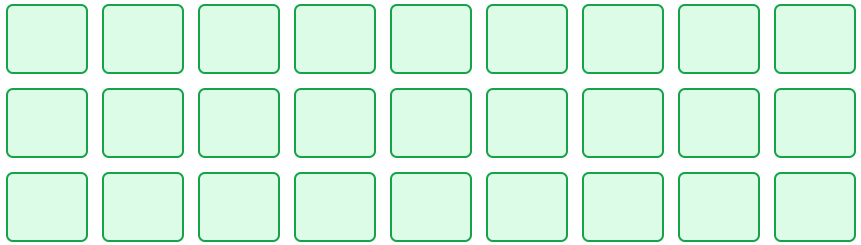
HumanEval / MBPP

Una función
con su docstring

Aprueban **> 90 %: agotado**

Sin dependencias ni archivos

SWE-bench Verified



Repositorio con miles de líneas

Aprueban **40 % a 55 %**

Contenedor completo con suite de pruebas

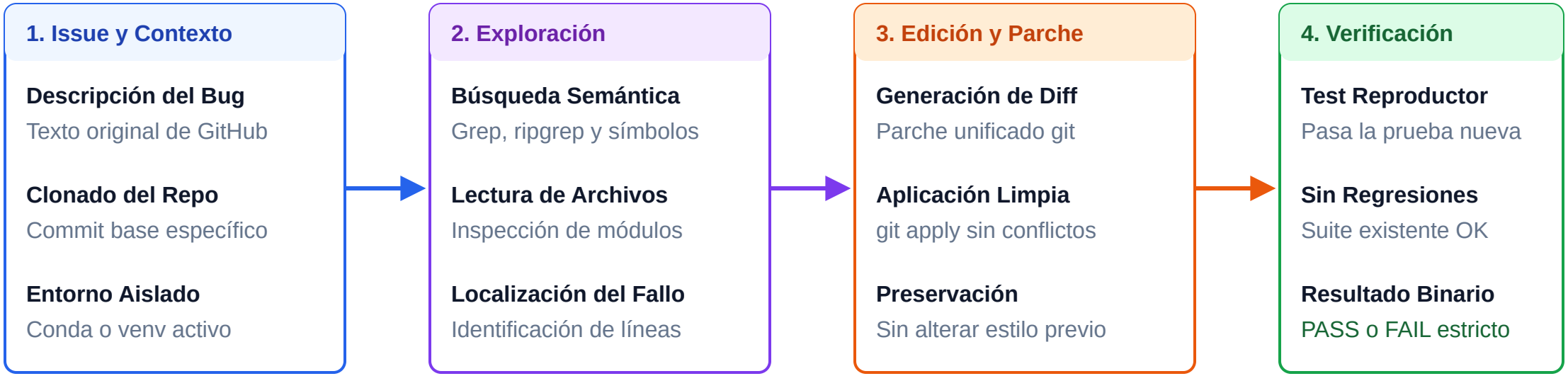
Un benchmark sirve mientras todavía separa a los modelos

Fuente: Princeton NLP — SWE-bench: Can Language Models Resolve Real-World GitHub Issues? (2024)

MECANISMO DE EVALUACIÓN

Ciclo de Resolución en SWE-bench

Flujo de trabajo autónomo desde la recepción del issue hasta la validación de tests



Fuente: Princeton NLP — SWE-bench Evaluation Harness Architecture (2024)

Indexación Semántica y Árboles Sintácticos

De búsqueda ingenua de texto a navegación por grafo de símbolos y dependencias



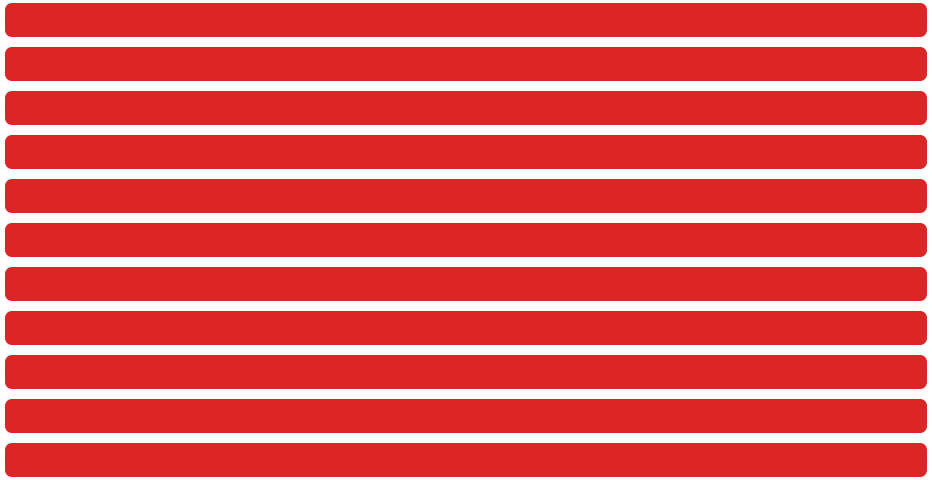
Dar al modelo menos texto, pero el correcto

Fuente: GitHub Next / Tree-sitter — Semantic Code Search Architecture (2024)

Edición Quirúrgica vs Reescritura Completa

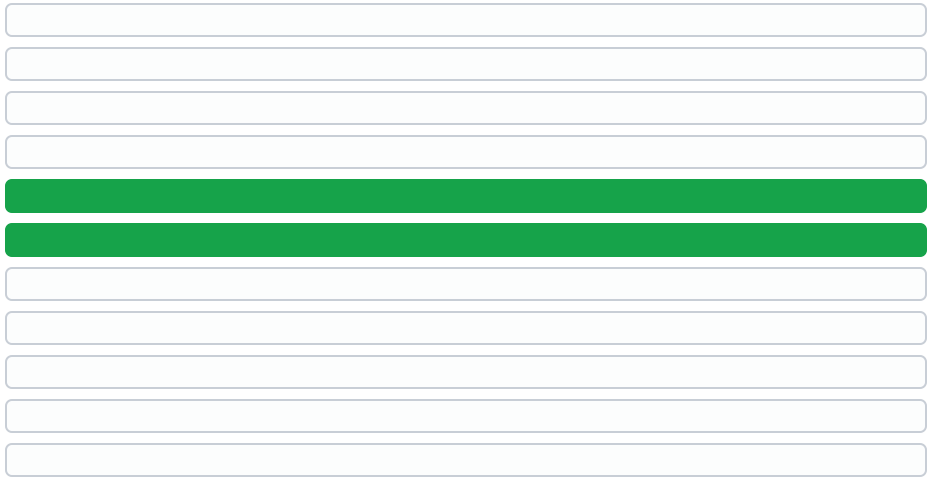
Minimizar tokens generados y evitar corrupción de archivos grandes

Reescritura completa



Miles de líneas y riesgo de truncar

Bloque SEARCH / REPLACE



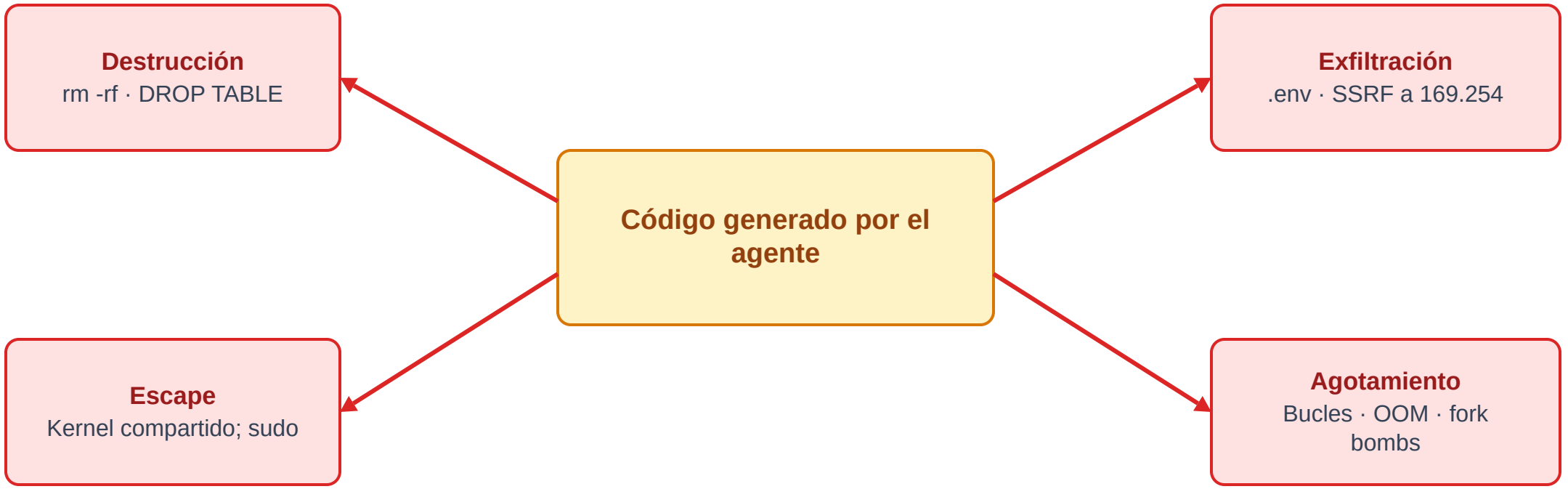
5 a 15 líneas, con coincidencia exacta

Se ahorra salida y se conserva intacto lo que no cambia

Fuente: Aider.chat / Anthropic — Code Editing Benchmarks and Formats (2024)

El Riesgo de Código No Confiable en la Nube

Vectores de ataque al permitir que modelos ejecuten comandos o scripts en servidores



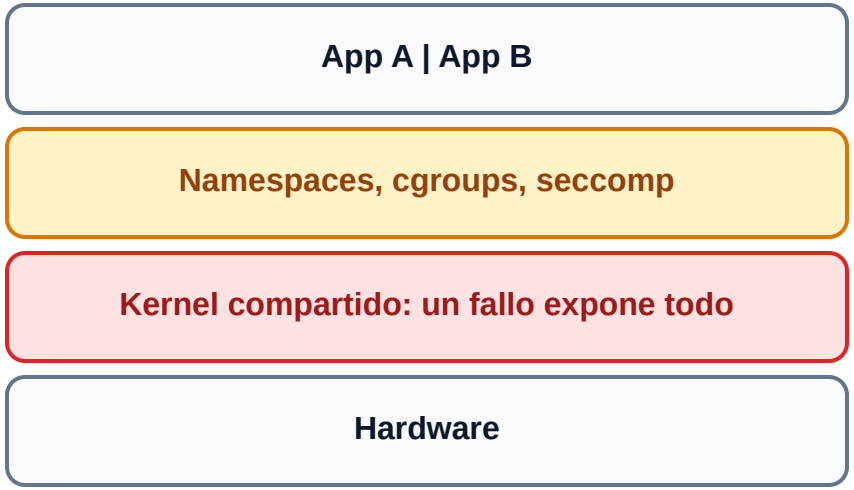
El código de un modelo se trata como el de un desconocido

Fuente: OWASP — Top 10 for Large Language Model Applications (2025)

Aislamiento: Contenedores vs MicroVMs

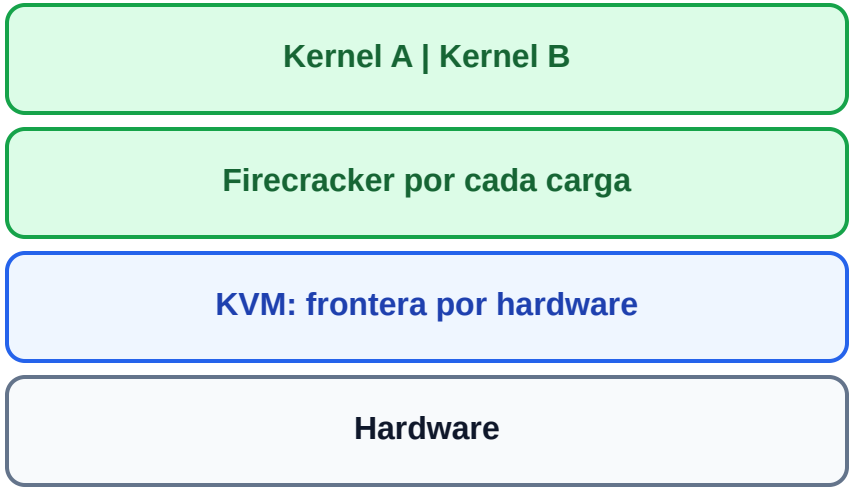
Comparativa entre namespaces de Linux y virtualización por hardware liviana

Contenedor



200 ms a 2 s de inicio

MicroVM



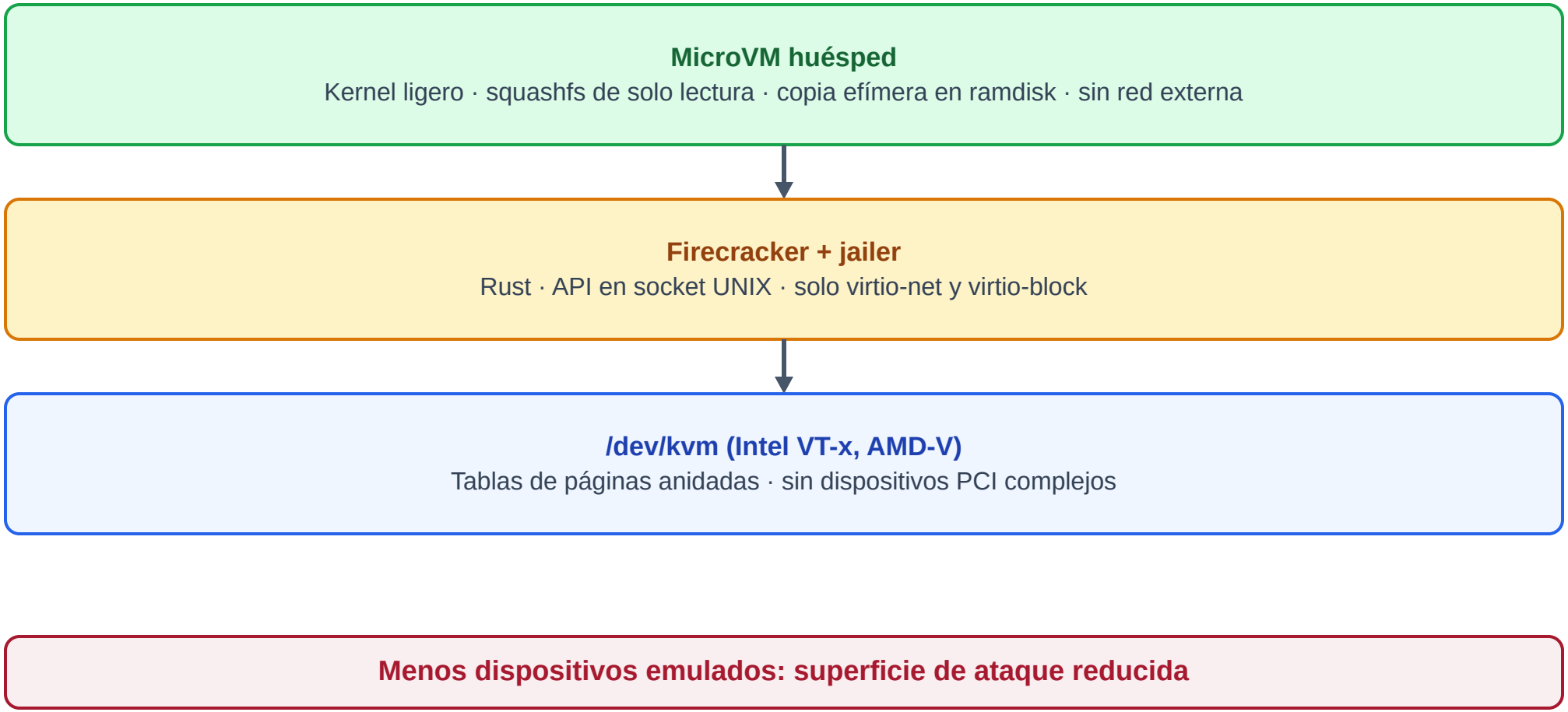
5 a 10 ms de arranque

La microVM da aislamiento fuerte casi sin pagar en tiempo de inicio

Fuente: AWS Open Source — Firecracker: Lightweight Virtualization for Serverless (2024)

Arquitectura de Firecracker MicroVM

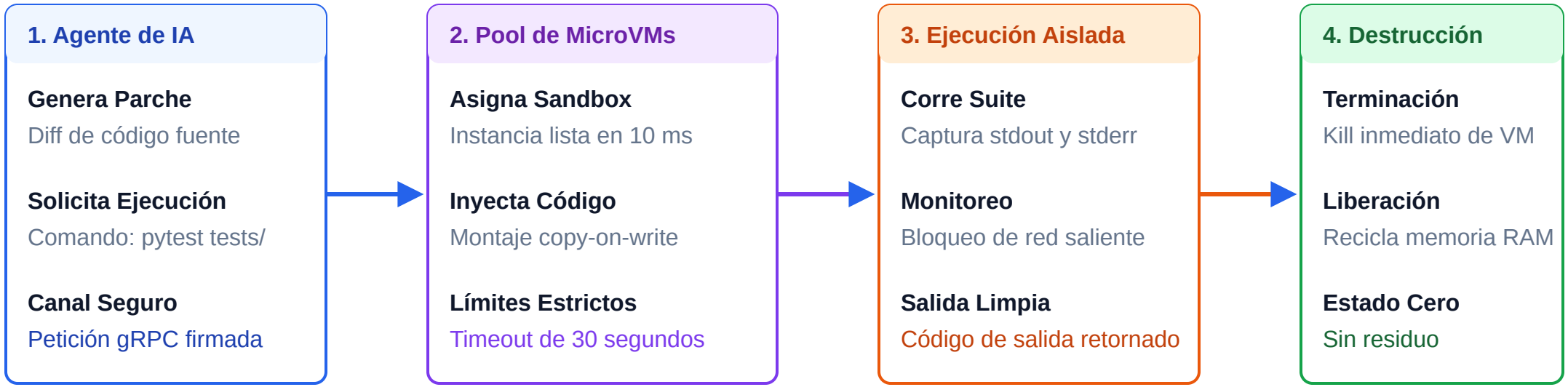
Monitor de máquinas virtuales minimalista escrito en Rust sobre KVM



Fuente: AWS Open Source — Firecracker Design and Implementation (2024)

Ciclo de Ejecución Segura en Nube

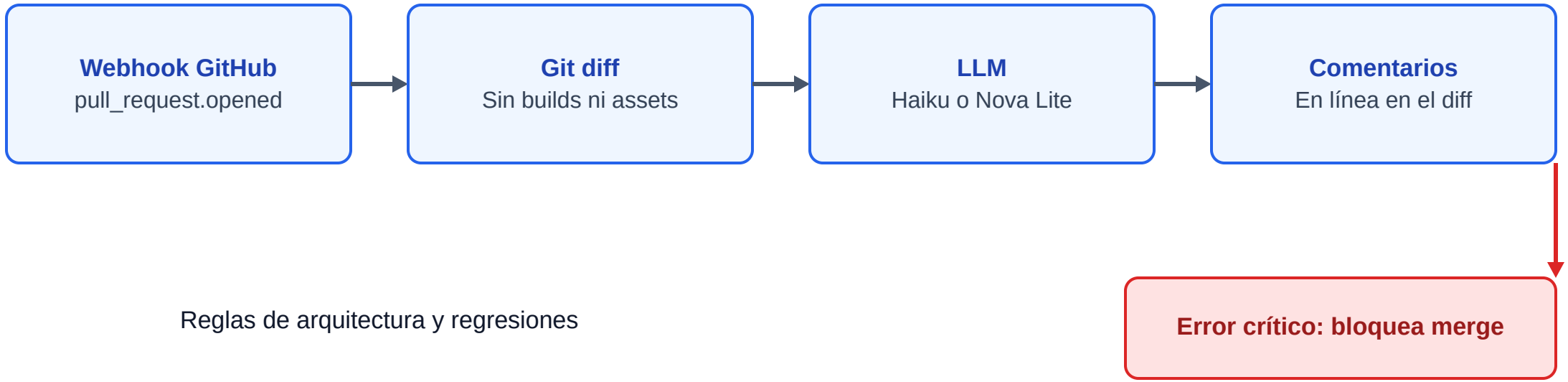
Flujo desacoplado: Agente de IA -> Orquestador de Sandboxes -> Resultado de Pruebas



Fuente: AWS Architecture Center — Secure Sandboxing for Generative AI (2024)

Revisión Asistida de Pull Requests en CI/CD

Agentes que analizan diffs, comentan fallas y proponen parches en el pipeline



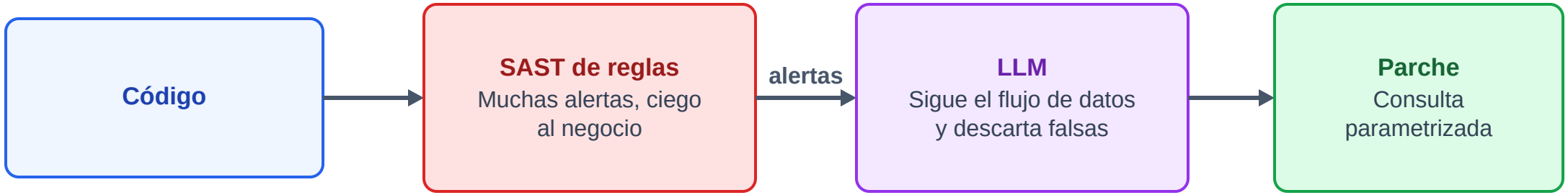
El modelo revisa cada PR; la decisión de aprobar sigue siendo del equipo

Fuente: GitHub / GitLab — Automated Code Review with AI in Pipelines (2024)

SAST Híbrido: Reglas Deterministas y Modelos

Complementar linters tradicionales con razonamiento semántico para eliminar falsos positivos

SAST híbrido: cada capa filtra a la siguiente

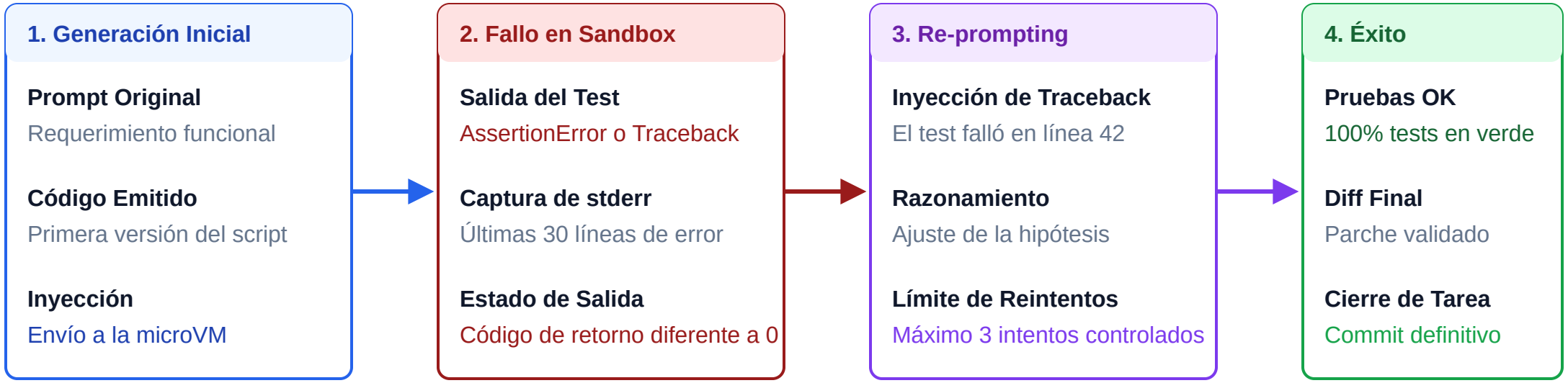


Las reglas no se saltan nada; el modelo les quita el ruido

Fuente: Snyk / SonarQube — AI-Assisted Vulnerability Detection Reports (2024)

Bucle de Retroalimentación de Error

Mecanismo autoreparador: Compilación/Test -> Error -> Re-prompting -> Parche

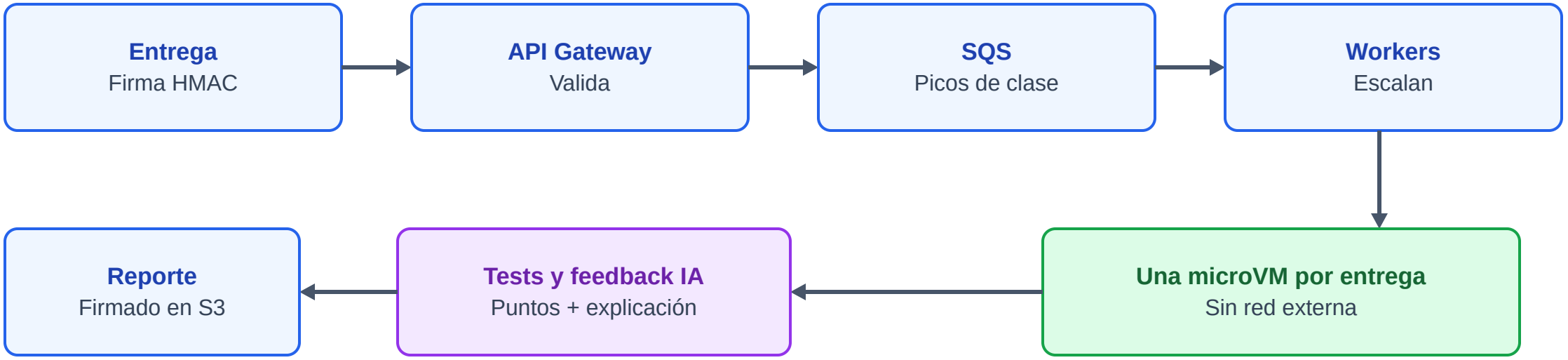


Fuente: Princeton NLP / SWE-bench Engineering Guide (2024)

CASO ILUSTRATIVO

Caso: Plataforma de Evaluación Automatizada de Código

Caso ilustrativo de arquitectura, no es una empresa real



Caso ilustrativo: el código del estudiante nunca toca el servidor

Fuente: Patrón de referencia de arquitectura de sandboxing para educación superior (2024)

RESUMEN EJECUTIVO

Síntesis de Parte 2: Principios de Código Seguro

Directrices esenciales de arquitectura para integración de modelos de desarrollo

1. Evaluación Rigurosa (SWE-bench)

Pruebas en Repositorios Reales

Descartar benchmarks simples como HumanEval para producción

Medir resolución de incidencias con suites de prueba completas

2. Edición Quirúrgica de Archivos

Bloques Search / Replace y Diffs

Nunca solicitar la reescritura total de archivos extensos

Ahorra hasta 80% de tokens de salida y evita truncamientos

3. Micro-Sandboxing Obligatorio

Fronteras Fuertes con Firecracker

Jamás ejecutar scripts generados en el host de producción

Aislamiento por virtualización KVM sin acceso a red externa

4. Bucle Autoreparador Controlado

Re-prompting con Tracebacks

Inyectar la salida de error del test para corregir el parche

Limitar estrictamente a 3 reintentos para controlar el gasto

Fuente: Guía de Arquitectura de Sistemas de IA en la Nube (2026)

SEMANA 16 — INTELIGENCIA ARTIFICIAL EN LA NUBE III

Sistemas Agénticos y Protocolo MCP

Parte 3 de 4 — Patrón ReAct, Invocación de Herramientas, Memoria y Gobernanza

Ing. Rodolfo Cañas Cervantes
Universidad de la Costa (CUC) — Ingeniería de Sistemas
Semana 16 — Periodo 2026-2 — Barranquilla, Colombia

Parte 3 de 4 — Patrón ReAct, Invocación de Herramientas, Memoria y Gobernanza

Del Chatbot Estático al Agente Autónomo

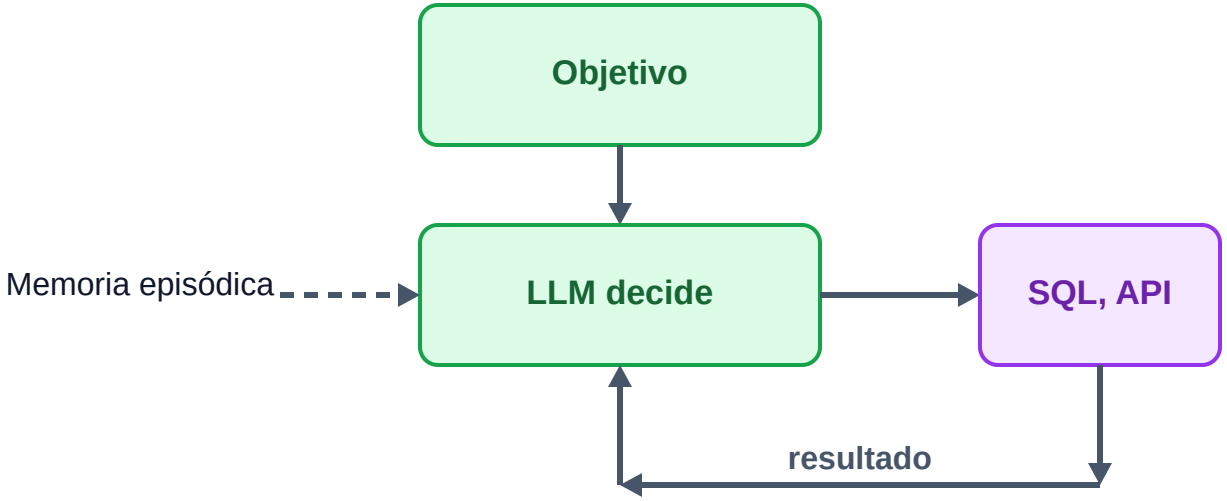
Integración de modelo fundacional, herramientas del entorno, memoria y bucle de decisión

Chatbot: entra y sale



Sin datos externos; conocimiento congelado

Agente: bucle con herramientas

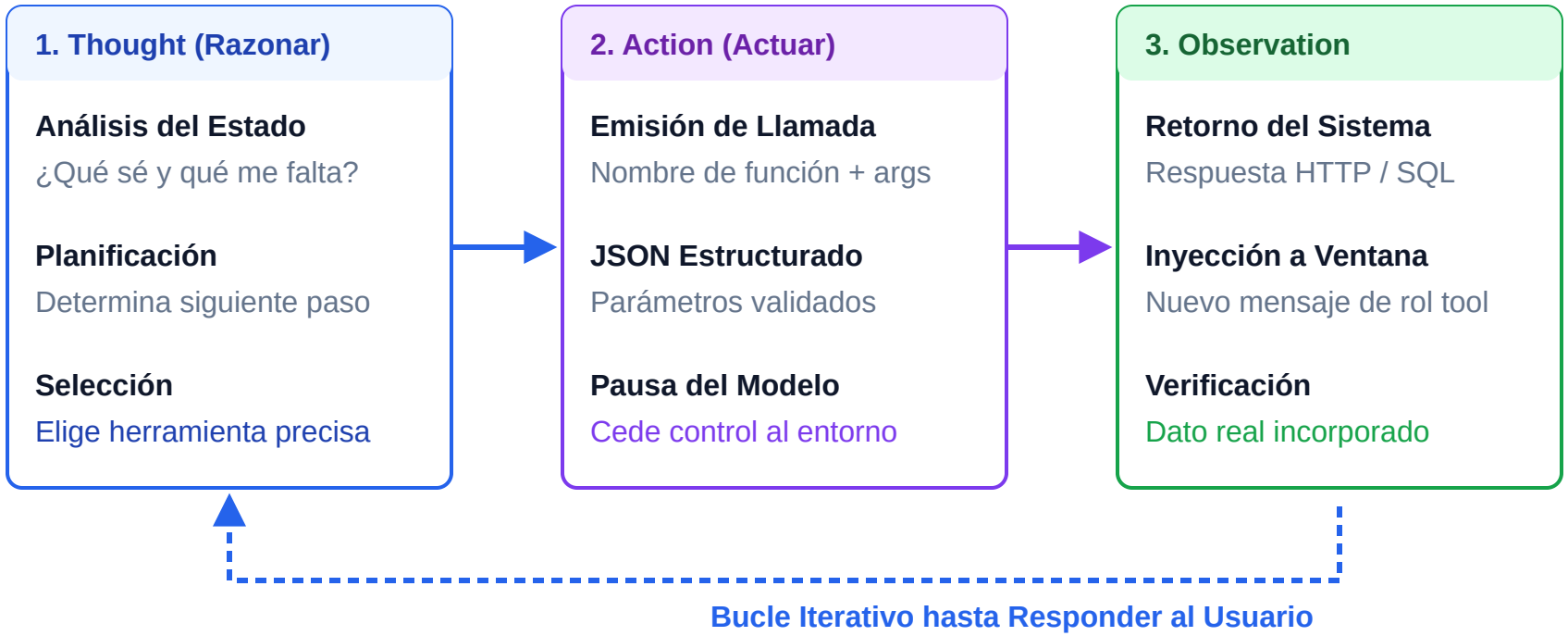


La diferencia no es el modelo: es que actúa y observa

Fuente: Anthropic — Building Effective Agents (Architectural Guide 2024)

El Patrón ReAct: Thought, Action, Observation

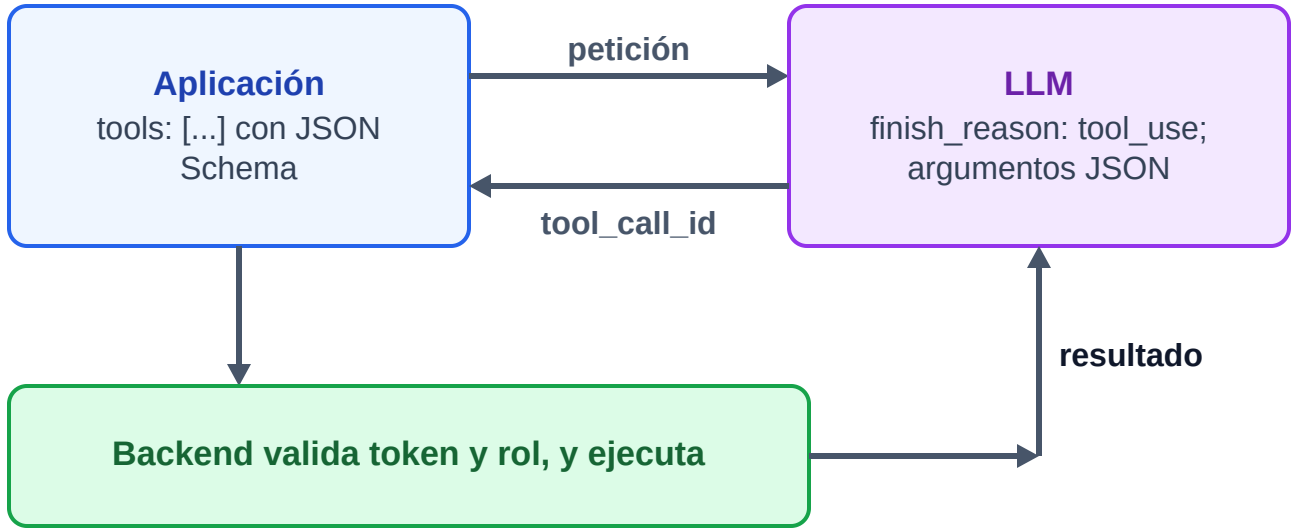
Bucle continuo de deliberación interna, ejecución instrumental y retroalimentación empírica



Fuente: Yao et al. — ReAct: Synergizing Reasoning and Acting in Language Models (ICLR 2023)

Mecanismo Técnico de Function Calling

Cómo el modelo pasa de lenguaje natural a generar estructuras de datos formales



El modelo no ejecuta nada

solo propone la llamada

Los permisos se comprueban en el backend, nunca en el prompt

Fuente: AWS Bedrock / OpenAI — Function Calling Specification (2024)

Diseño Riguroso de JSON Schema para Herramientas

Descripciones semánticas claras y tipado estricto para evitar alucinaciones en parámetros

Esquema ambiguo

run_query
query: string

↓

Inyección SQL, sintaxis alucinada

Esquema estricto

consultar_saldo_cuenta
cuenta_id: ^CTA-[0-9]{8}$ · tipo: enum

↓

Éxito > 98 %, sin llamadas espurias

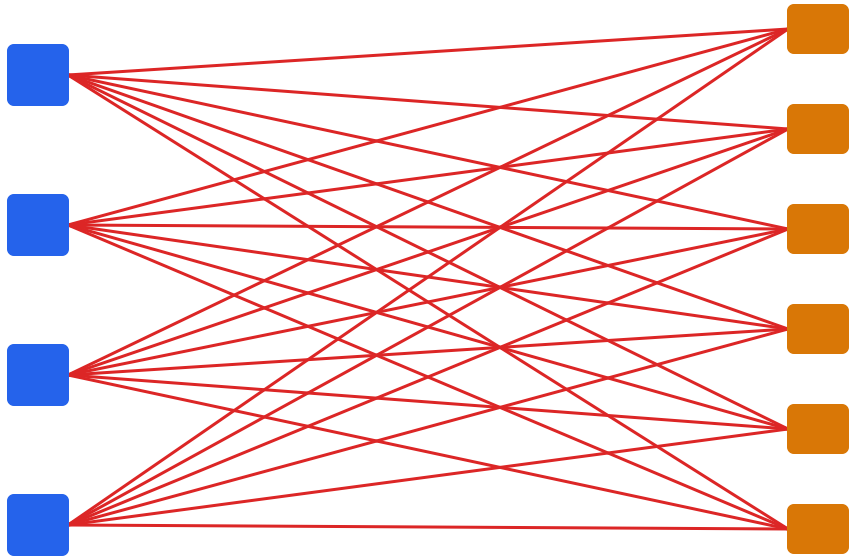
Un esquema estricto es la primera barrera contra errores

Fuente: JSON Schema Specification (Draft 2020-12) / Anthropic Tool Use Guide (2024)

Model Context Protocol (MCP): El Puerto USB-C de la IA

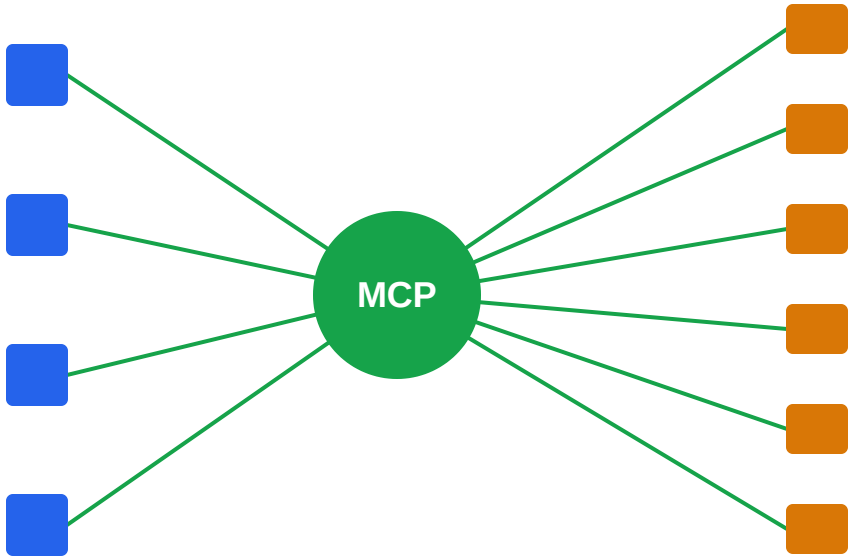
Estándar abierto para desacoplar modelos de fuentes de datos y servidores de herramientas

Punto a punto: N × M



4 × 6 = 24 conectores

MCP: N + M



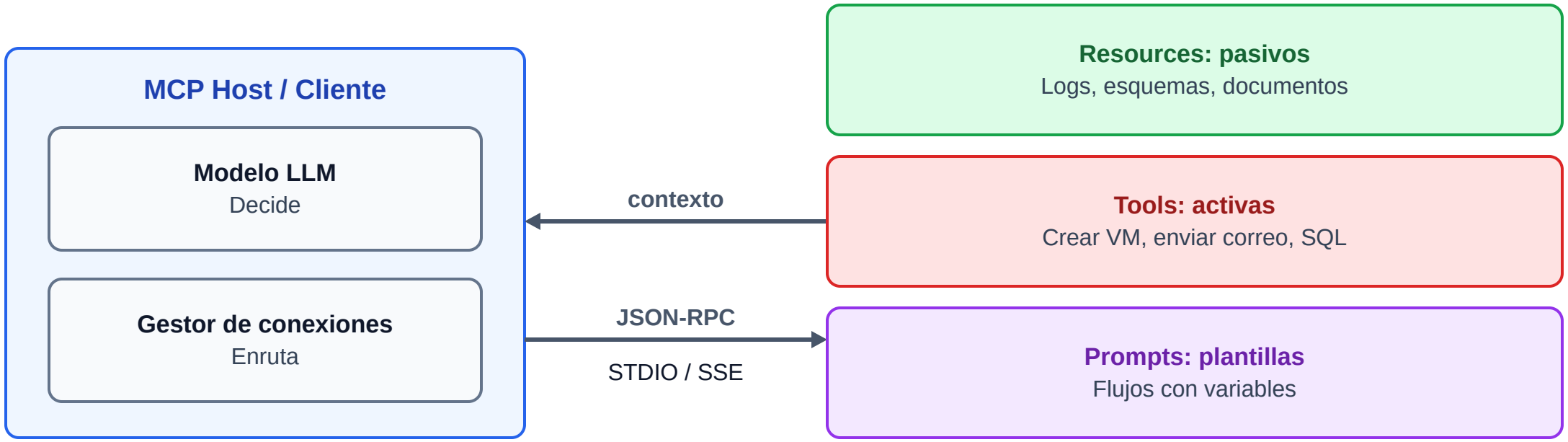
4 + 6 = 10 interfaces

Un servidor MCP sirve a cualquier cliente que hable el protocolo

Fuente: Anthropic — Model Context Protocol Specification (Nov 2024)

Arquitectura Cliente-Host-Servidor en MCP

Tres primitivas de exposición: Resources (datos), Tools (acciones) y Prompts (plantillas)



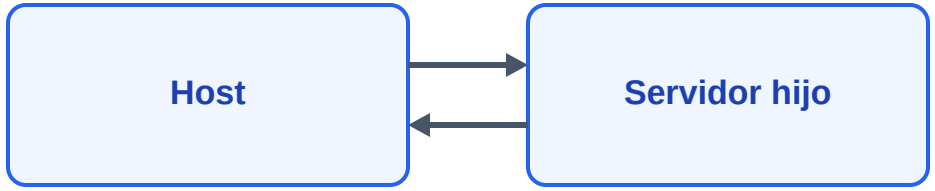
Separar lo que se lee de lo que actúa permite controlar cada riesgo

Fuente: Anthropic — Model Context Protocol Architecture (2024)

Mecanismos de Transporte MCP: STDIO vs SSE

De procesos hijos en el host local a microservicios distribuidos en la nube

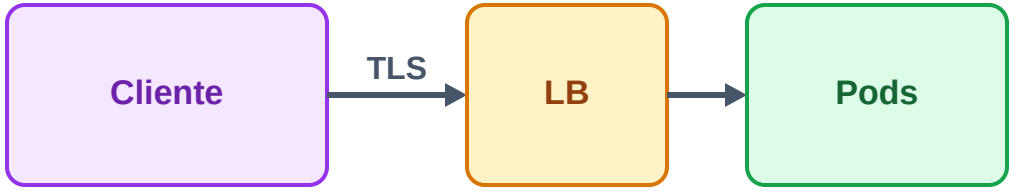
STDIO: local



stdin / stdout, sin puertos

Hereda permisos del usuario

SSE + HTTP: remoto



POST para peticiones, SSE para streaming

Exige mTLS, OAuth 2.0 y RBAC

Al salir de la máquina local, la seguridad pasa a ser tu trabajo

Fuente: Model Context Protocol — Transports and Security Specification (2024)

Jerarquía de Memoria en Sistemas Agénticos

De buffers volátiles en contexto a almacenes vectoriales y relacionales persistentes

Trabajo: ventana de contexto
Inmediata · volátil

Episódica: pgvector u OpenSearch
15 a 50 ms · incidentes y tareas previas

Semántica: PostgreSQL o DynamoDB
Consulta por clave · perfiles y reglas de negocio

Más arriba: más rápida y más escasa. Más abajo: persiste entre sesiones

Fuente: CoALA Architecture — Cognitive Architectures for Language Agents (Sumers et al. 2024)

Compactación y Ventana Deslizante de Contexto

Estrategias para sostener sesiones largas de agente sin agotar la memoria del modelo

Ventana llena: dos maneras de hacer espacio

FIFO



Olvida el prompt de sistema: deriva

Compactar



Conserva hechos con 80 % menos tokens

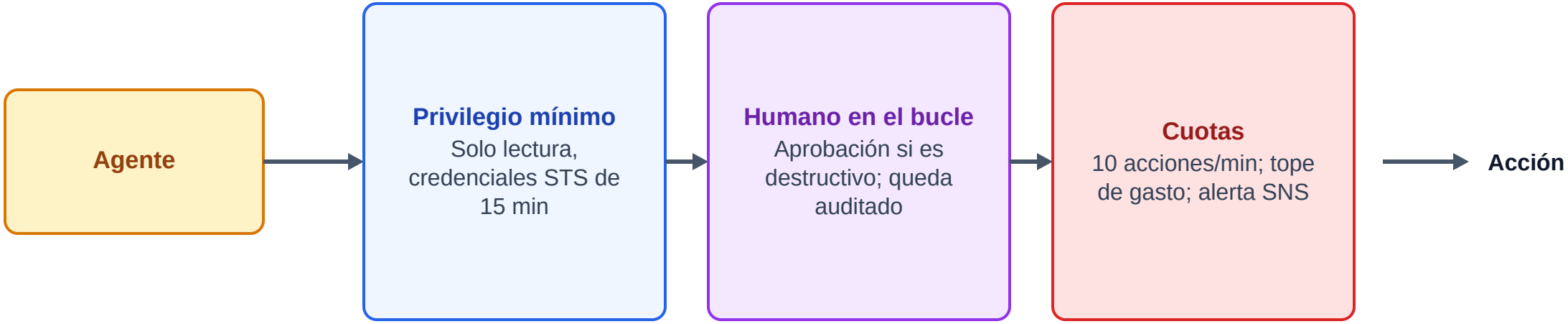
S = prompt de sistema: las reglas fundamentales nunca se eliminan

Fuente: Anthropic / OpenAI — Long-Running Agent Sessions Guide (2024)

Acotando el Radio de Impacto (Blast Radius)

Principio de mínimo privilegio, credenciales con alcance limitado y aprobaciones humanas

Una acción del agente pasa por tres barreras

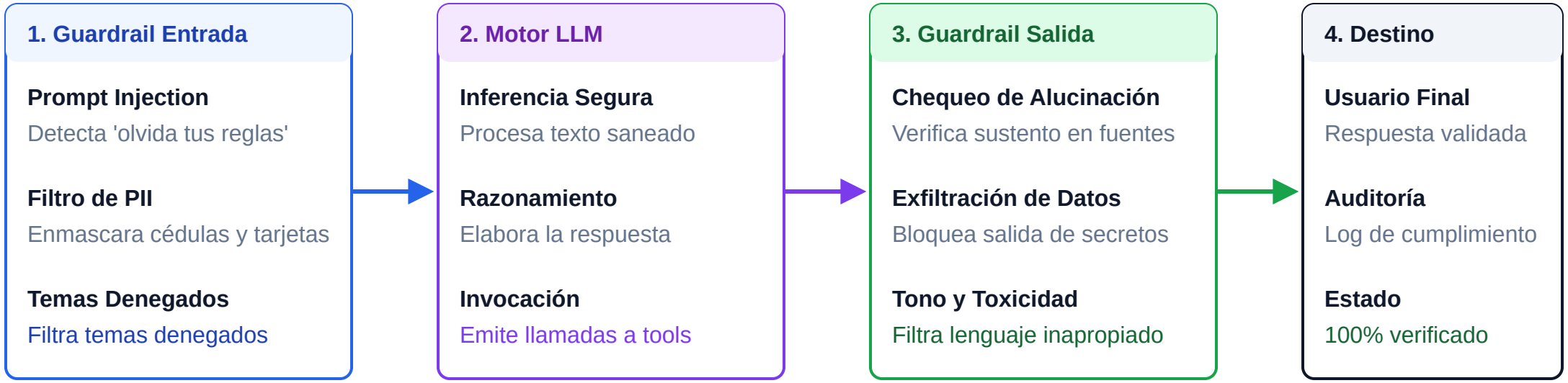


Si el agente falla, el daño queda acotado

Fuente: AWS Security Best Practices for Generative AI Applications (2024)

Guardrails: Filtros de Entrada y Salida

Capas de validación deterministas y semánticas antes y después del modelo

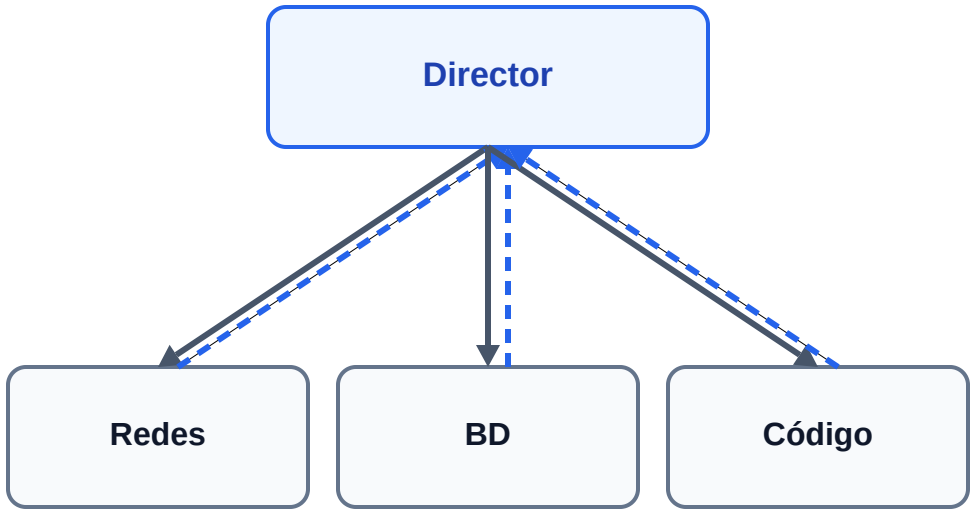


Fuente: AWS Bedrock Guardrails / NeMo Guardrails Documentation (2024)

Patrones de Orquestación Multi-Agente

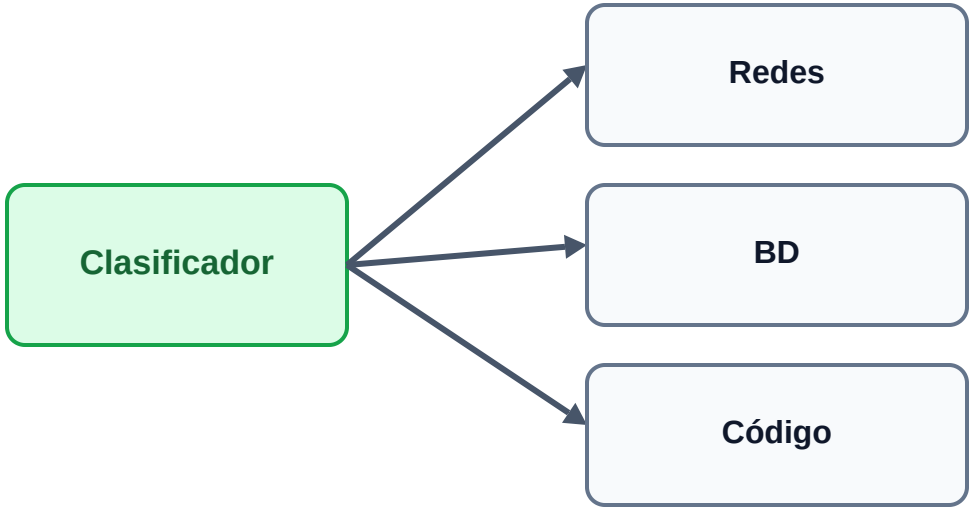
De agentes solitarios a equipos cooperativos: Supervisor central vs Enrutamiento dinámico

Supervisor



Predecible y auditable

Enrutamiento y handoff



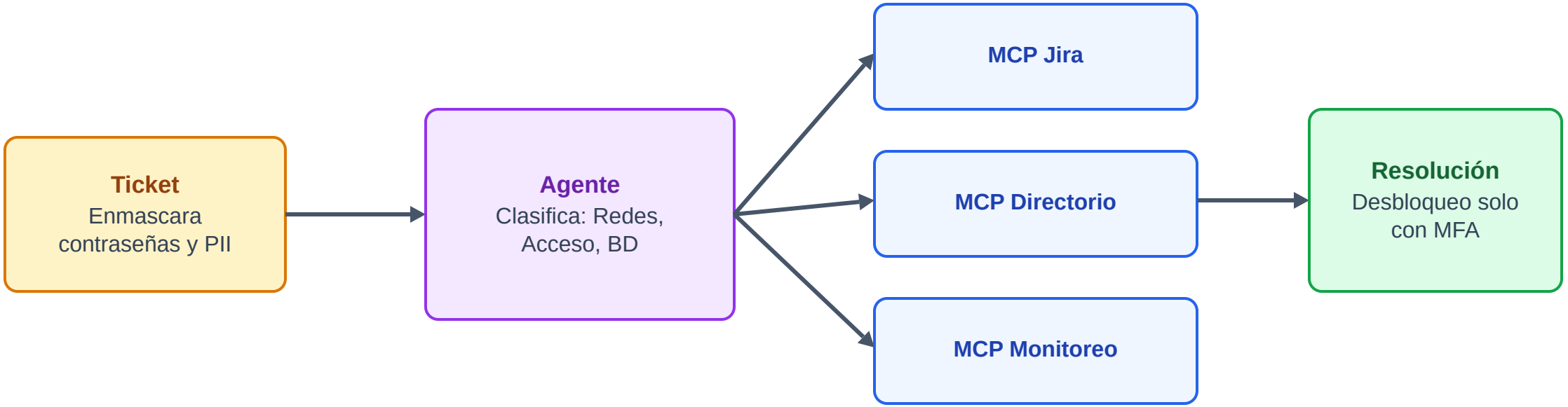
Menor latencia y menos tokens

Control o eficiencia: el patrón se elige según lo que hay que auditar

Fuente: LangChain / AutoGen — Multi-Agent Systems Architectural Patterns (2024)

Caso: Mesa de Ayuda Agéntica con Servidores MCP

Caso ilustrativo de arquitectura, no es una empresa real



Caso ilustrativo: las herramientas cambian; el agente no

Fuente: Patrón de referencia para mesa de ayuda asistida por IA y MCP (2024)

RESUMEN EJECUTIVO

Síntesis de Parte 3: Principios de Sistemas Agénticos

Reglas de oro para construir y desplegar agentes confiables en la nube

1. Patrón ReAct y JSON Schema Estricto

Contratos de Datos Inmutables

Tipar parámetros con expresiones regulares y enumeraciones

El backend valida permisos antes de ejecutar cualquier acción

2. Adopción de Model Context Protocol (MCP)

Desacoplamiento Estándar

Separar datos (Resources), acciones (Tools) y plantillas (Prompts)

Evolución de STDIO local a microservicios distribuidos con SSE

3. Jerarquía y Compactación de Memoria

Gestión de Contexto Sostenible

Memoria de trabajo, vectores y BD

Compactar mediante resúmenes sin descartar el prompt de sistema

4. Blast Radius y Guardrails Estrictos

Seguridad y Defensa en Profundidad

Tokens efímeros, cuotas de gasto y aprobación humana obligatoria

Filtros de prompt injection al entrar y anti-fugas al salir

SEMANA 16 — INTELIGENCIA ARTIFICIAL EN LA NUBE III

Repaso Integral de la Unidad 3

Parte 4 de 4 — Síntesis de Nubes Privadas, Cómputo GPU, Inferencia y Agentes

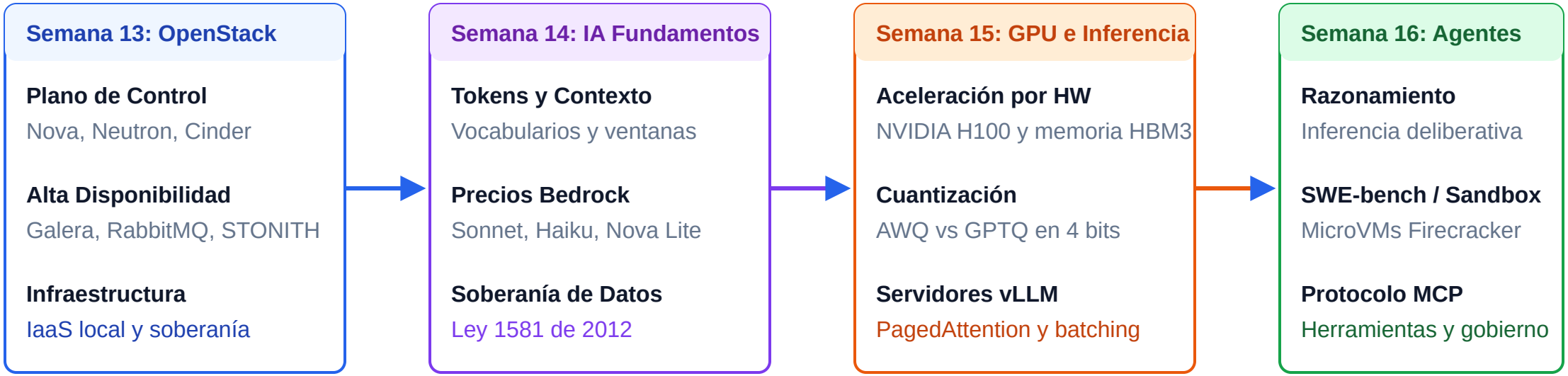
Ing. Rodolfo Cañas Cervantes
Universidad de la Costa (CUC) — Ingeniería de Sistemas
Semana 16 — Periodo 2026-2 — Barranquilla, Colombia

Parte 4 de 4 — Síntesis de Nubes Privadas, Cómputo GPU, Inferencia y Agentes

VISIÓN HOLÍSTICA

Mapa de Ruta: De la Infraestructura Base a la IA

Cuatro semanas articuladas: Nube privada, inferencia básica, aceleración y sistemas agénticos



Fuente: Silabo de Arquitectura en la Nube — Universidad de la Costa (2026)

OpenStack: Ciclo de Vida de Creación de una VM

Secuencia coordinada: Keystone -> Nova API -> Glance -> Neutron -> Cinder -> Nova Compute

Secuencia de Interacción entre Servicios OpenStack

1. Autenticación

Keystone

Valida usuario

Emite token Fernet

Catálogo endpoints

Token verificado

2. Entrada Nova

Nova API / Conductor

Verifica cuotas

Registra registro BD

Nova Scheduler

Elige hipervisor

3. Red y Disco

Neutron y Cinder

Neutron reserva IP

Crea puerto virtual

Cinder adjunta LUN

Recursos listos

4. Imagen SO

Glance

Descarga qcow2/raw

Transfiere a nodo

Copia efímera

Imagen en caché

5. Hipervisor

Nova Compute

Invoca libvirt / KVM

Enchufa vNIC a OVN

Arranque del guest

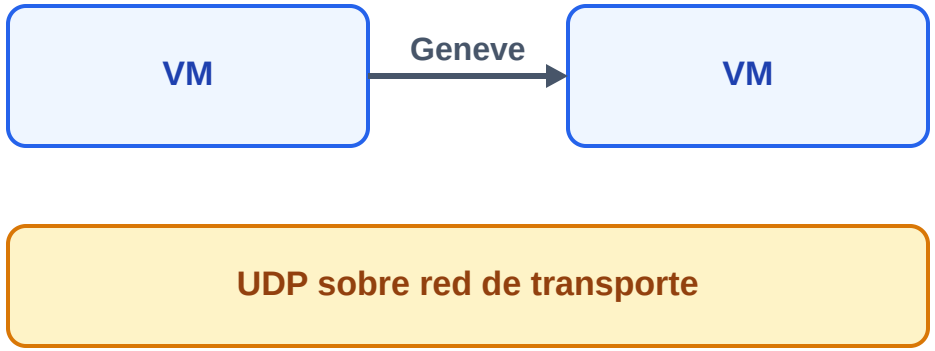
Estado: ACTIVE

Fuente: OpenStack Architecture Guide (2024.1 Caracal)

Redes de Proyecto (Overlay) vs Proveedor (Flat/VLAN)

Aislamiento multi-inquilino con túneles Geneve frente a conectividad física directa

Red de proyecto (overlay)



El usuario define sus rangos

Salida: router virtual y Floating IP

Red de proveedor (Flat/VLAN)



MTU 1500, sin encapsular

Solo los administradores las crean

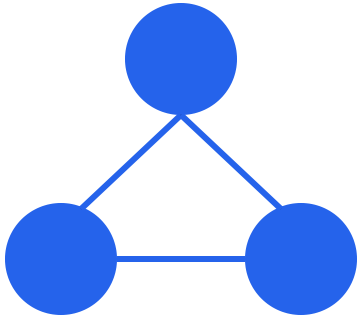
Autonomía del proyecto frente a integración directa con la red física

Fuente: OpenStack Neutron Documentation — Networking Concepts (2024)

Alta Disponibilidad: Galera, RabbitMQ y STONITH

Tolerancia a fallos en el plano de control: Quórum de base de datos, colas espejo y esgrima de nodos

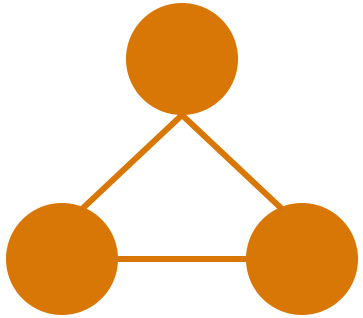
Galera



Réplica síncrona

Minoría se desconecta

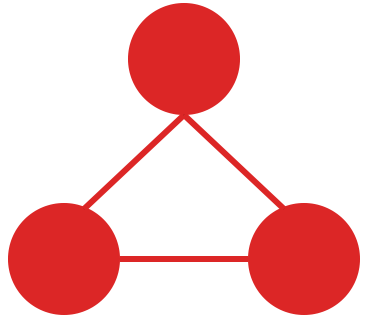
RabbitMQ



Bus RPC central

Quorum Queues en 3 nodos

Pacemaker + STONITH



Heartbeat de Corosync

Fencing e IP virtual flotante

Tres nodos: dos siempre pueden formar mayoría y aislar al tercero

Fuente: OpenStack High Availability Guide (2024.2)

Hardware de Aceleración: GPUs y Memoria HBM3

Superar el cuello de botella de memoria: Ancho de banda extremo e interconexiones de baja latencia



Por eso el acelerador gana en LLM: cada token exige leer los pesos

Tensor Cores: FP8 y FP16 por hardware

NVLink 4: 900 GB/s entre GPU

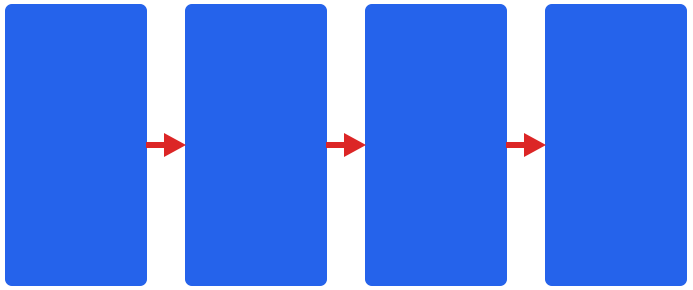
Ancho de banda de memoria unas 10 veces superior

Fuente: NVIDIA H100 Tensor Core GPU Architecture Whitepaper (2024)

Cuantización: AWQ vs GPTQ en Inferencia

Reducción de memoria a 4 bits protegiendo los pesos salientes del modelo

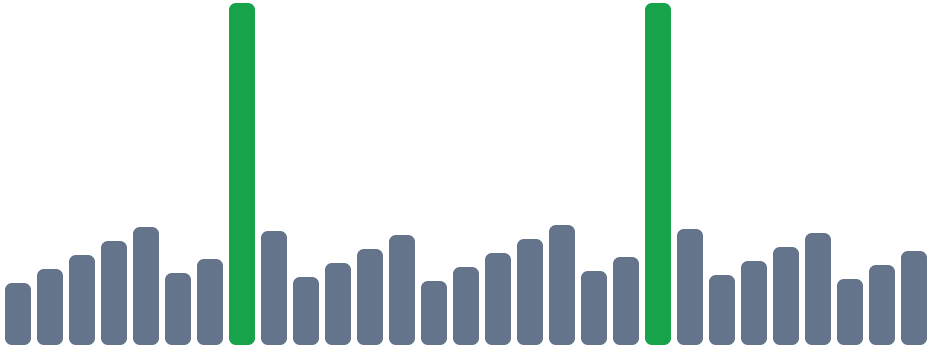
GPTQ: compensa por capas



Ajusta el resto para absorber el error

Bueno en lotes pequeños mono-usuario

AWQ: protege lo saliente



Protege el 1 % sensible antes de INT4

Optimizado en vLLM y TensorRT-LLM

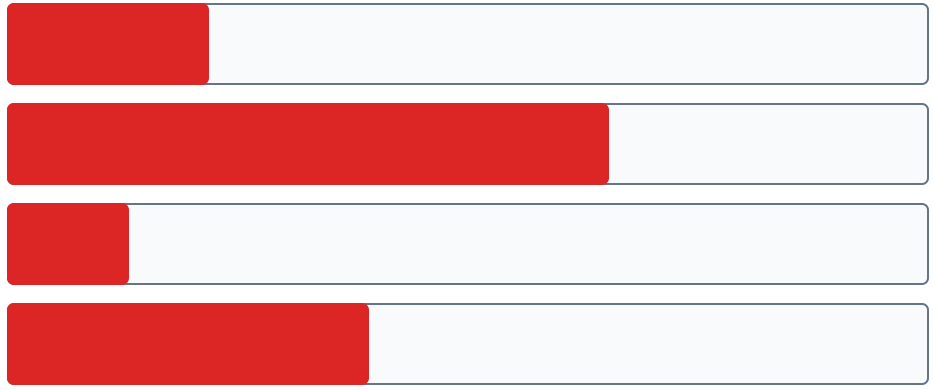
Repaso: uno repara el error, el otro evita cometerlo donde más duele

Fuente: Lin et al. — AWQ: Activation-aware Weight Quantization (MLSys 2024)

vLLM: PagedAttention y Batching Continuo

Eliminación de la fragmentación de memoria y saturación óptima del cómputo GPU

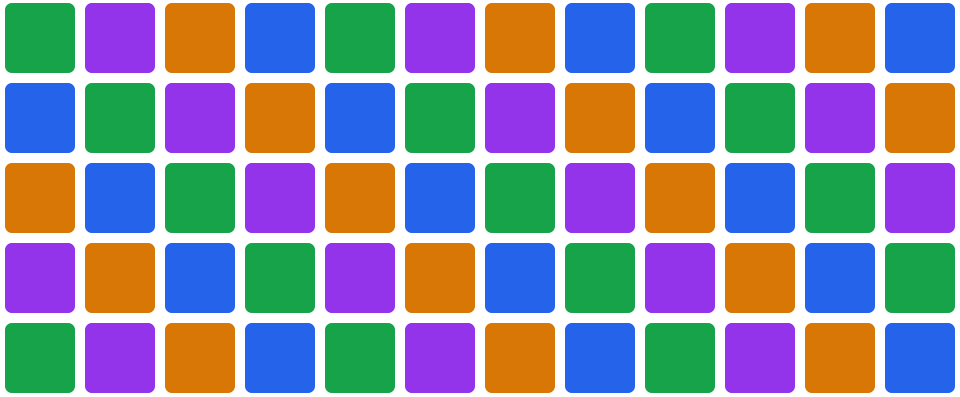
Tradicional



Reserva por max_context_length

60 % a 80 % bloqueado; lote rígido

vLLM



Páginas de 16 tokens, < 4 % de desperdicio

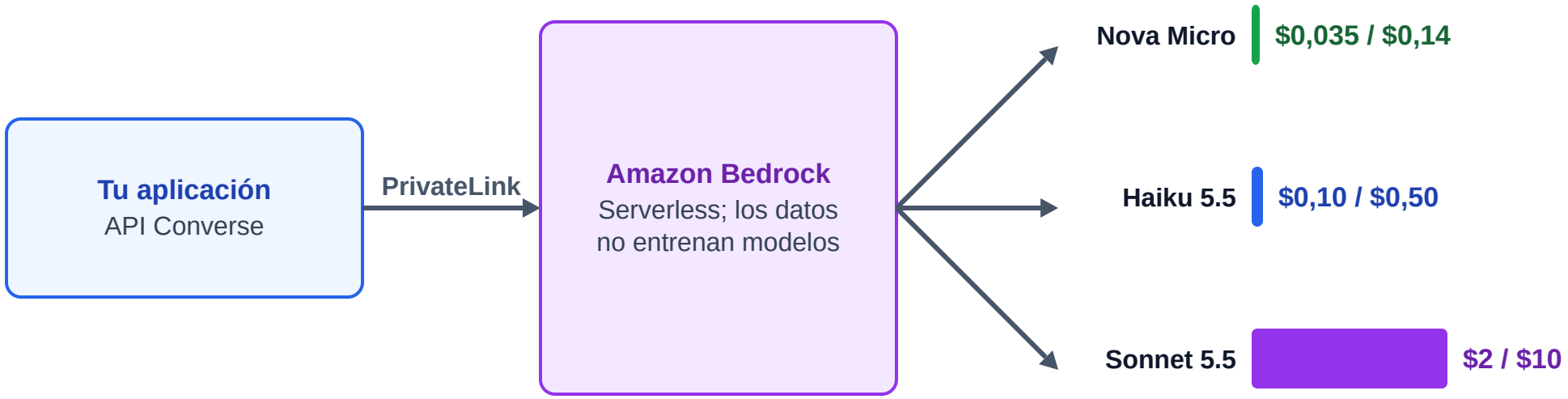
Nuevas peticiones entran en cada token

Repaso: hasta 4 veces más usuarios por GPU

Fuente: Kwon et al. — Efficient Memory Management for LLM Serving with PagedAttention (SOSP 2023)

Amazon Bedrock: Catálogo Multimodelo Serverless

Inferencia elástica bajo demanda sin gestión de servidores, clusters ni GPUs físicas



Entrada / salida por 1M de tokens, nov 2026

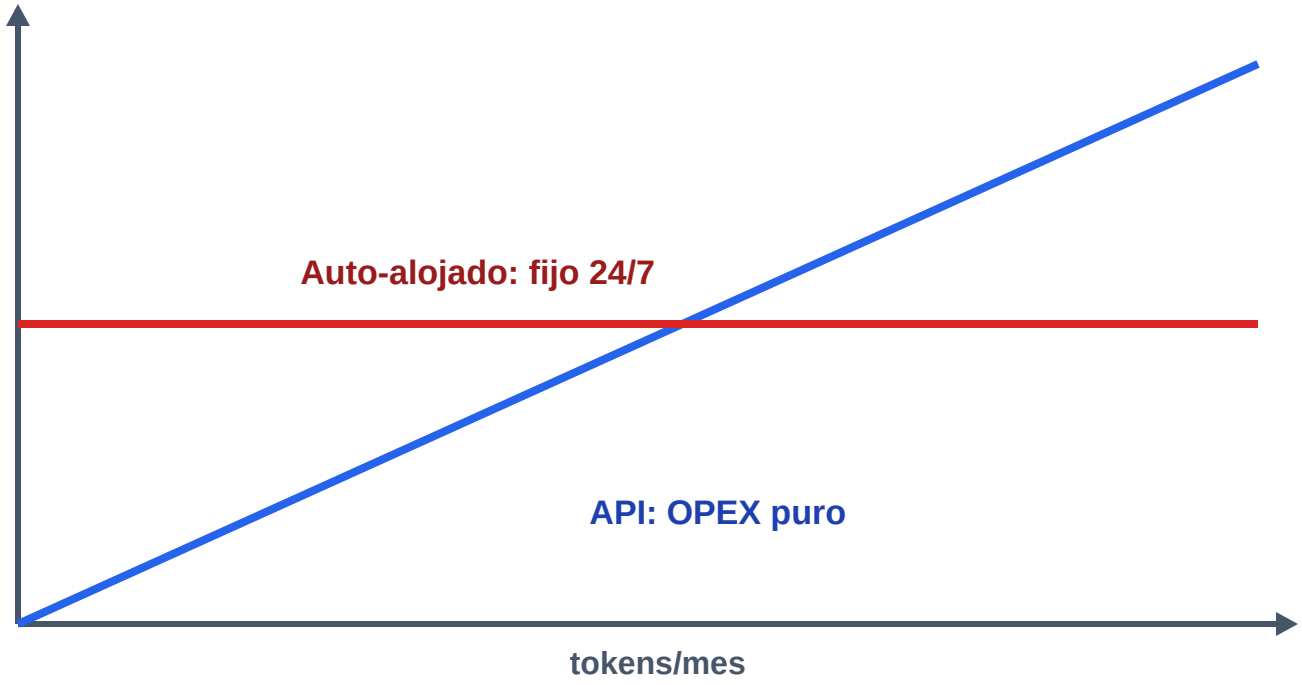
Un solo código, varios modelos: la caché reduce aún más la lectura

Fuente: AWS Bedrock Documentation & Official Pricing (Consultado Noviembre 2026)

Evaluación de TCO: ¿Cuándo Auto-Alojar vs API?

Análisis del punto de equilibrio entre costos fijos de infraestructura y costos variables por token

Dos estructuras de costo



API
Prototipos, tráfico variable, mínima operación

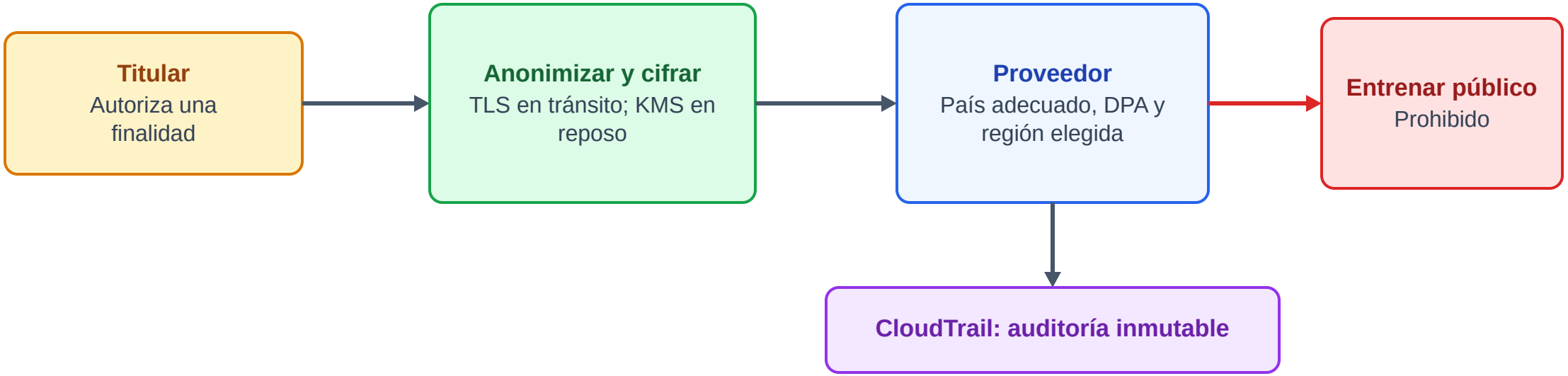
Auto-alojado
Tráfico masivo o soberanía; requiere MLOps

Calcular con el tráfico real y sumar el equipo de operación

Fuente: AWS Cloud Economics — Total Cost of Ownership for LLMs (2024)

Soberanía de Datos y Cumplimiento: Ley 1581

Requisitos legales en Colombia para procesamiento de datos personales en la nube



Cada tramo responde a un principio de la Ley 1581

Fuente: Superintendencia de Industria y Comercio (SIC) — Guías de Cumplimiento Ley 1581 (2024)

GUÍA DE DECISIÓN

Matriz de Selección: ¿Qué Tecnología Elegir?

Árbol de criterios según latencia, presupuesto, volumen de datos y criticidad de seguridad

Criterios de Elección Tecnológica en la Unidad 3

OpenStack Local

Soberanía Estricta
Datos que no pueden salir del campus

Control de Hardware
Sin costos variables

IaaS bajo tu control

Amazon Bedrock

Velocidad al Mercado
Despliegue inmediato sin comprar servidores

Costo Flexible
Pagas solo por token

Inferencia Serverless

Clúster vLLM + AWQ

Alto Rendimiento
Rendimiento predecible sin límites de cuota

Volumen Masivo
Costo fijo eficiente

Auto-alojamiento top

Agentes con MCP

Automatización Real
Ejecución con guardrail y sandboxing seguro

Ecosistema Abierto
Herramientas modulares

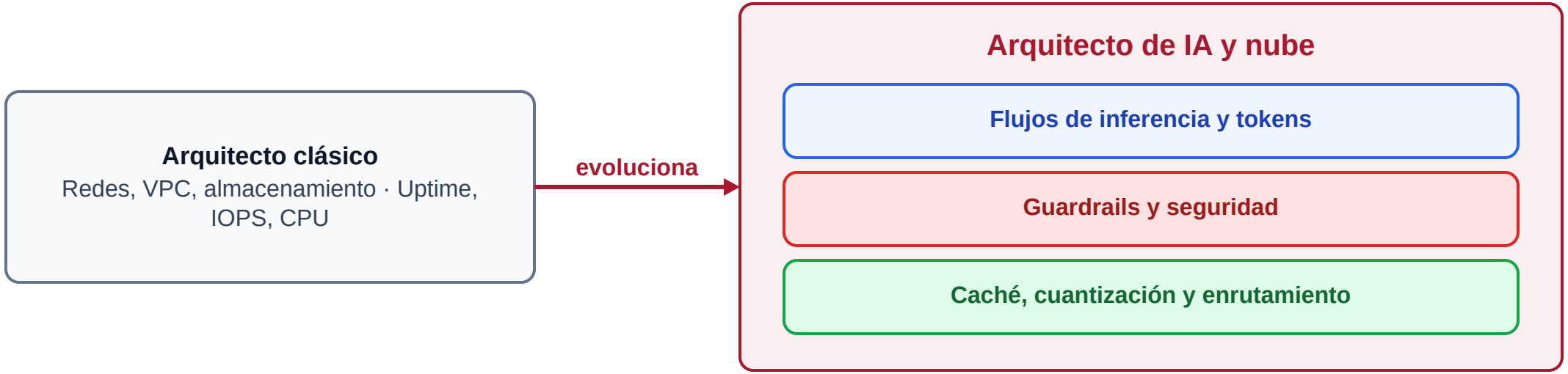
Delegación confiable

Fuente: Guía de Arquitectura de Sistemas de IA en la Nube (2026)

PERFIL PROFESIONAL

El Arquitecto de Soluciones en la Era de la IA

De simple aprovisionador de máquinas virtuales a diseñador de sistemas cognitivos confiables



La aplicación dejó de ser una caja negra para quien diseña la infraestructura

Fuente: AWS Certified Solutions Architect Professional Framework (2026)

CHECKLIST DE ENTREGA

Checklist de Arquitectura: Listos para Producción

Seis preguntas obligatorias antes de poner en marcha un sistema de IA en la nube

1. ¿Está acotado el radio de impacto de las herramientas?

Privilegios mínimos y esquemas tipados

2. ¿Se ejecuta el código en sandboxes aislados?

MicroVMs Firecracker efímeras sin acceso a red externa ni host

3. ¿Existe un tope presupuestal y cuotas por usuario?

Límites de tokens y cortacircuitos

4. ¿Operan guardrails de entrada y salida?

Detección inyección y filtro de PII

5. ¿Se cumple el marco de la Ley 1581 de datos?

Autorización, DPA firmado y cifrado

6. ¿Está justificada la elección entre API y Auto-alojamiento?

TCO validado según volumen y equipo

Fuente: AWS Well-Architected Framework — Generative AI Lens (2024)

Conclusión de la Unidad 3: El Futuro de la Nube

Integración armónica entre infraestructura sólida, modelos deliberativos y gobernanza responsable

Tres Pilares Inmutables de la Arquitectura en la Nube

1. Fundamentos Sólidos

Infraestructura Resiliente

Redes OVN y alta disponibilidad

Almacenamiento y quórum Galera

Hardware GPU y enlaces HBM3

Sin base, ningún LLM escala

2. Eficiencia y Escala

Optimización de Recursos

Cuantización AWQ en 4 bits

PagedAttention y batching en vLLM

Serverless en Amazon Bedrock

Máximo valor por dólar invertido

3. Gobierno y Autonomía

Sistemas Agénticos Seguros

Protocolo abierto MCP

Micro-sandboxes con Firecracker

Cumplimiento estricto Ley 1581

IA confiable y ética al servicio